# Capital Gains Banking Website Project Documentation

System overview, file guide, database schema, user flow and implementation report

Prepared from the three project phases in this workspace. The final PHP phase is described as written in source code; the website and database were not executed for this report.

## Executive summary

Capital Gains is an educational banking website prototype. A visitor can view the welcome and about pages, register an account, log in, view account details and loan history, submit a loan request, and initiate an account to account transfer. The project progresses from static pages in phase 1, through expanded HTML form rules and an unfinished validation script in phase 2, to a PHP and MariaDB implementation in phase 3.

Its purpose is to demonstrate how a web interface, server side logic, PHP sessions and a relational database work together for common banking tasks. The current implementation is suitable for learning and demonstration, not for real financial use.

## Project phases

| Phase | Contents | What it contributes |
|---|---|---|
| 1 | HTML pages and images | Navigation, page layout and forms without database behavior. |
| 2 | HTML pages and JavaScript | HTML form patterns; an unconnected validation function and empty login script. Data is not persisted. |
| 3 | PHP pages, CSS and bank.sql | MySQL connection, session login, registration, loans and transfer updates. |

## Why the main parts exist

The browser displays forms and sends POST requests. PHP receives the submitted fields, reads or changes rows through MySQLi, and stores the logged in account identifiers in a session. MariaDB stores the customers, accounts, branches, employees and loans. Bootstrap supplies page styling and interactive navigation; local images and animation.css provide the project branding and animations.

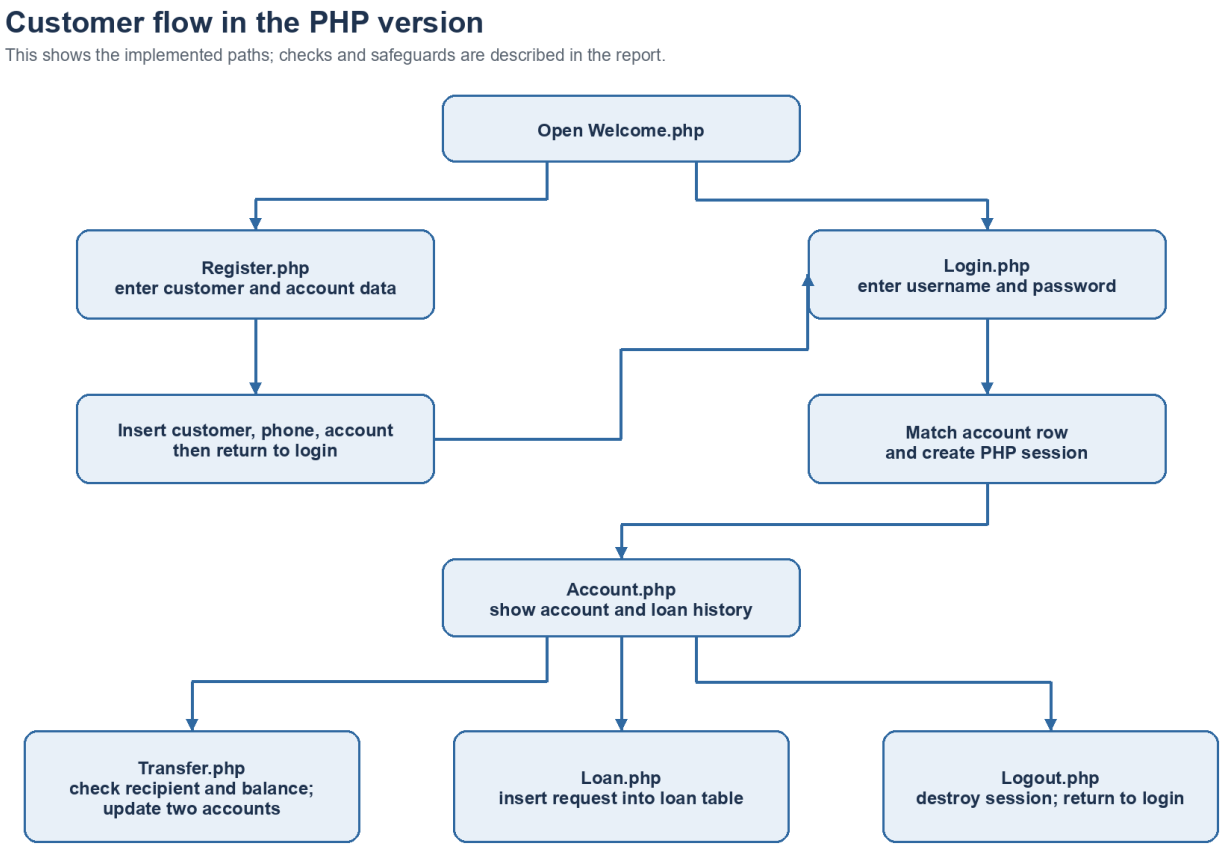



Figure 1. Customer journey through the PHP pages. Arrows describe navigation and main writes, not every error branch.

## How a customer task works

**Registration.** Register.php reads customer, contact, account and branch fields. It queries for an existing username or email, then inserts customer, customer_phone_numbers and account rows if no match is found. A new account starts with a zero balance and the page redirects to login after a successful multi query.

**Login.** Login.php searches account for the submitted username and password. If exactly one row matches, it stores the username, account number, customer SSN, account type, balance and branch ID in the PHP session, marks loggedin true, and redirects to Account.php.

**Account view.** Account.php reads loan rows for the customer SSN in the session. It shows the username, account number, account type and balance from session values and lists loans in a table. The displayed balance is a session snapshot rather than a fresh account query.

**Transfer.** Transfer.php takes the source account from the session, looks up the entered recipient account, checks that recipient differs from source, then compares the source balance with the amount. It sends two UPDATE statements to debit and credit the accounts and changes the session balance after a successful multi query.

**Loan request.** Loan.php takes the customer SSN and branch ID from the session and inserts the amount, type, dates and user selected interest into loan. There is no approval or repayment calculation in this page.

**Logout.** Logout.php clears and destroys the PHP session and redirects to Login.php.

## What each file does

| File or folder | Responsibility |
|---|---|
| Welcome.php | Public home page, image carousel and external exchange rate widget. |
| About.php | Project/team information and a description of offered features. |
| Register.php | Registration form and inserts for three database tables. |
| Login.php | Account credential lookup and session creation. |
| Account.php | Account summary from session and loan history from database. |
| Transfer.php | Recipient lookup and balance updates. |
| Loan.php | Loan request form and loan row insertion. |
| Logout.php | Session termination and login redirect. |
| conn.php | MySQLi connection to the local bank database. |
| bank.sql | Schema, sample rows, indexes and foreign keys. |
| animation.css and images/ | Visual styling and image assets. |

## Database scheme

The SQL dump defines six tables. Account and loan connect customer activity to a branch. Customer phone numbers allow multiple phone rows per customer. Employee records connect staff to a branch and optionally to a supervising employee. The application pages use account, customer, customer_phone_numbers and loan; branch is used through branch ID, while employee is represented in the schema but not used by the PHP pages.

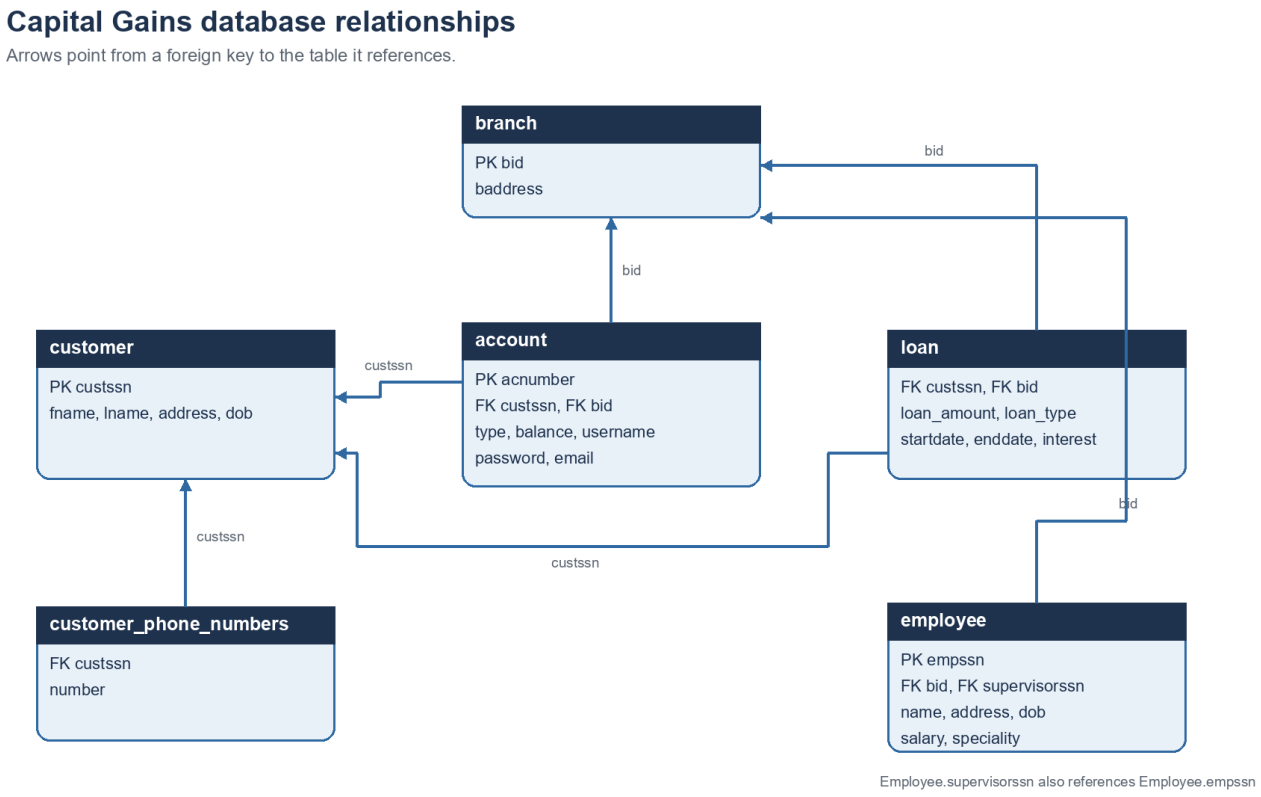



Figure 2. Relationships represented by the foreign keys in bank.sql. Loan has no declared primary key in this dump.

| Table | Key fields | Role |
|---|---|---|
| customer | custssn (PK) | Person identity and address/date of birth. |
| account | acnumber (PK); custssn and bid (FK) | Account type, balance, login fields and customer/branch links. |
| customer_phone_numbers | custssn (FK); number | Additional customer phone numbers. |
| branch | bid (PK) | Branch address; sample data names Alexandria and Cairo. |
| loan | custssn and bid (FK) | Loan amount, type, dates and interest. |
| employee | empssn (PK); bid and supervisorssn (FK) | Staff details, branch and supervisor relationship. |

## Local setup from the source

Use a local PHP installation with the MySQLi extension and MariaDB/MySQL, such as a XAMPP style environment. Create a database named bank, then import Projectphase3/Project/Project/bank.sql. Place the contents of that phase's inner Project directory in the web server document root and open Welcome.php through the local server. conn.php currently expects a local MySQL server, a root user, an empty password and the bank database; those settings must match the local environment. Bootstrap and the home page exchange rate widget need internet access.

The SQL file contains CREATE TABLE statements but no CREATE DATABASE statement, so the bank database must exist before import. Treat its sample rows as demonstration data.

## Implementation findings and limits

- **Passwords and queries.** Passwords are compared and stored as plain text, including sample account rows in bank.sql. Queries use string interpolation instead of prepared statements, which exposes registration, login, loan and transfer operations to SQL injection.
- **Access control.** Account.php, Loan.php and Transfer.php start a session and alter navigation based on loggedin, but their main page and POST logic do not first enforce an authenticated session. Direct requests can reach those paths.
- **Transfer integrity.** The debit and credit are sent as two statements without an explicit database transaction or row locking. A partial update or concurrent change could leave incorrect balances. The amount is not checked as a positive numeric value, and the strict greater than comparison rejects a transfer equal to the available balance.
- **Loan semantics.** A loan request is inserted immediately, with the interest rate selected by the applicant. There is no approval state, payment plan or loan disbursement. The loan table has no primary key.
- **History and stale values.** The About page mentions transaction history, but there is no transaction table or transfer history page. Account.php displays loan history; the balance comes from the login session and can become stale after other changes.
- **Registration integrity.** Registration uses three INSERT statements through multi_query without a transaction. If a later insert fails, earlier data may remain. The duplicate account message condition compares submitted strings with isset results and can fail to show the alert. Register.php also reads session navigation state without starting a session.
- **Personal data and configuration.** The SQL dump contains sample personal details and credentials. conn.php embeds local database connection settings. These should be replaced and protected before any sharing or deployment.

## Suggested next steps

For a stronger learning project, first add server side authentication checks to every private page and POST action, prepared statements, password hashing, and validation of amounts and dates. Then make transfers atomic in a database transaction, record each transfer in a transaction table, and refresh balances from the database. A loan status and approval workflow would make the loan feature match its wording. Finally, move connection settings outside source and replace the sample personal data before publishing.

## Source and verification note

This report is based on reading the phase folders and the PHP, JavaScript and SQL source files in this workspace on 15 October 2026. It documents intended and coded behavior. No live PHP or database execution was performed, so runtime behavior and complete security impact were not verified.